РАСКРЫТИЕ ПРОБЛЕМЫ МНОГОКРАТНОГО ПОРОЖДЕНИЯ И УНИЧТОЖЕНИЯ НИТЕЙ

МАЙСТРЕНКО МАРАТ 323 СКИ ВМК МГУ

ПЛАН ДОКЛАДА

- Постановка проблемы
- Неправильный код
- Исправление кода
- Измерение ускорения. Анализ результатов
- Заключение

ПОСТАНОВКА ПРОБЛЕМЫ

- Многократное порождение и уничтожение потоков в OpenMP может привести к значительным накладным расходам на управление потоками
- Эти накладные расходы могут существенно снизить общую производительность программы, особенно если параллельные секции выполняются часто

НЕПРАВИЛЬНЫЙ КОД

- Это приведет к тому, что значительная часть времени выполнения программы будет **потрачена** не на полезные вычисления, а **на управление потоками.**

- На каждой итерации:
  Создаются новые потоки
  После функции потоки уничтожаются.

```cpp
#include <iostream>
#include <vector>
#include <omp.h>
#include <cstdlib>
#include <ctime>

using namespace std;

void matrixVectorMultiply(const vector<vector<double>>& matrix, const vector<double>& vector) {
    size_t rows = matrix.size();
    size_t cols = matrix[0].size();
    vector<double> result(rows, 0.0);

    #pragma omp parallel for
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < cols; ++j)
            result[i] += matrix[i][j] * vector[j];
}

int main() {
    const size_t size = 1000; // Размер матрицы и вектора
    const int iterations = 100; // Количество итераций умножения

    vector<vector<double>> matrix(size, vector<double>(size));
    vector<double> vec(size);

    srand(42);
    for (size_t i = 0; i < size; ++i) {
        for (size_t j = 0; j < size; ++j)
            matrix[i][j] = static_cast<double>(rand()) / RAND_MAX;
        vec[i] = static_cast<double>(rand()) / RAND_MAX;
    }

    for (int i = 0; i < iterations; ++i)
        vector<double> result = matrixVectorMultiply(matrix, vec);

    return 0;
}
```

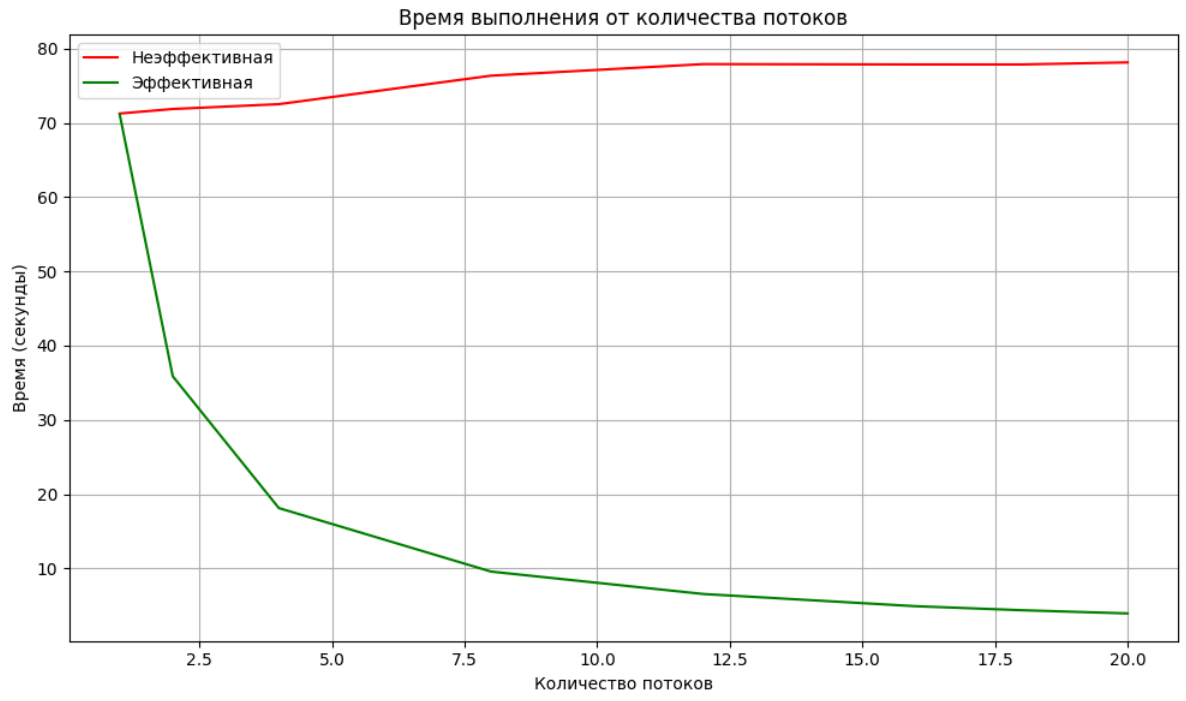
ИСПРАВЛЕНИЕ КОДА

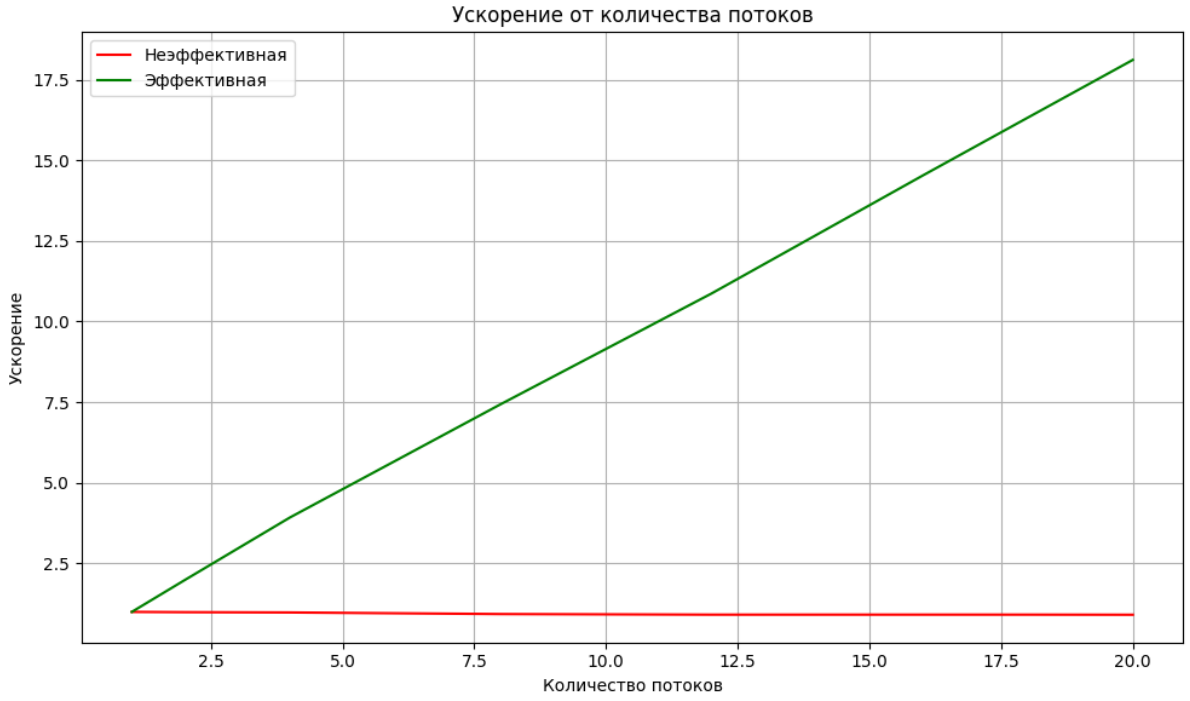
- Чтобы устранить проблему, нужно **вынести параллельную секцию за пределы цикла**. Это позволит создать потоки один раз и использовать их для всех итераций

- Таким образом, боремся с:
  Накладными расходами
  Снижением производительности
  Неэффективным использованием ресурсов

```cpp
#include <iostream>
#include <vector>
#include <omp.h>
#include <cstdlib>

using namespace std;

void matrixVectorMultiply(const vector<vector<double>>& matrix, const vector<double>& vec) {
    size_t rows = matrix.size();
    size_t cols = matrix[0].size();
    vector<double> result(rows, 0.0);

    for (size_t i = 0; i < rows; i++)
        for (size_t j = 0; j < cols; j++)
            result[i] += matrix[i][j] * vec[j];
}

int main() {
    const size_t size = 1000; // Размер матрицы и вектора
    const int iterations = 100; // Количество итераций умножения

    vector<vector<double>> matrix(size, vector<double>(size));
    vector<double> vec(size);

    for (size_t i = 0; i < size; ++i) {
        for (size_t j = 0; j < size; ++j)
            matrix[i][j] = static_cast<double>(rand()) / RAND_MAX;
        vec[i] = static_cast<double>(rand()) / RAND_MAX;
    }

    #pragma omp parallel for // Потоки создаются один раз, итерации распределяются
    for (int i = 0; i < iterations; ++i)
        matrixVectorMultiply(matrix, vec);

    return 0;
}
```

ИЗМЕРЕНИЕ УСКОРЕНИЯ И АНАЛИЗ РЕЗУЛЬТАТОВ





- Исследование наглядно демонстрирует важность правильного использования потоков в параллельных вычислениях.
- Однократное создание потоков позволяет достичь значительного ускорения работы программы, в то время как многократное создание и уничтожение потоков приводит к деградации производительности.